

File Edit View Navigate Code Refactor Run Tools PythonProject1 Version Control Current File Start Free Trial

Project

- app.py
- char.txt
- doct.py
- Doctor.py
- Employee.py
- FamilyDoctor.py
- halla
- halla.txt
- lesson 15.py
- lesson 16.py
- lesson 17.py
- lesson 18.py
- lesson 19.py
- lesson 20.py
- lesson 21.py
- lesson 22.py
- lesson 23.py
- lesson 24.py
- lesson 25.py
- lesson 26.py
- lesson 27.py
- main.py
- network_reachability_ch

stu.py Employee.py Doctor.py doct.py FamilyDoctor.py ping pong.py oop-python.py xo.py new.py network_reachability_checker.py

```
4 def calculate_ospf_path(graph, start_router, end_router): 1 usage
26     heapq.heappush(priorities, (distance, neighbor))
27
28     return distances[end_router]
29
30
31 # تمثيل شبكة (Topology) مكونة من 4 راوترات
32 # الأرقام تمثل الـ Cost (مثلاً: سرعة اللينك)
33 network_topology = {
34     'Router_A': {'Router_B': 10, 'Router_C': 2},
35     'Router_B': {'Router_D': 5},
36     'Router_C': {'Router_B': 3, 'Router_D': 15},
37     'Router_D': {}
38 }
39
40 start = 'Router_A'
41 destination = 'Router_D'
42
43 best_cost = calculate_ospf_path(network_topology, start, destination)
44
45 print(f"--- محاكاة بروتوكول OSPF ---")
46 print(f"من {start} إلى {destination} هي: {best_cost} (Cost) أقل تكلفة")
```

Run network_reachability_checker

C:\Users\ha\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\ha\PycharmProjects\PythonProject1\network_reachability_checker.py

--- محاكاة بروتوكول OSPF ---

أفضل تكلفة (Cost) من Router_A إلى Router_D هي: 10

Process finished with exit code 0

PythonProject1 > network_reachability_checker.py 46:20 (4 chars) LF UTF-8 4 spaces Python 3.14 (PythonProject1)

ENG 30°C غائم جزئياً screenrec

اكتب هنا للبحث

Employee.py Doctor.py doct.py FamilyDoctor.py ping pong.py oop-python.py xo.py new.py network_reachability_checker.py

```
import heapq

def calculate_ospf_path(graph, start_router, end_router):
    # قاموس لتخزين أقل تكلفة للوصول لكل راوتر
    distances = {router: float('infinity') for router in graph}
    distances[start_router] = 0

    # قائمة انتظار الأولويات (Priority Queue)
    priority_queue = [(0, start_router)]

    while priority_queue:
        current_distance, current_router = heapq.heappop(priority_queue)

        # إذا وصلنا للهدف، نتوقف
        if current_router == end_router:
            return current_distance

        # البحث في الروابط المجاورة (Neighbors)
        for neighbor, cost in graph[current_router].items():
            distance = current_distance + cost

            # إذا وجدنا مساراً بتكلفة أقل، نقوم بتحديثه
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(priority_queue, item: (distance, neighbor))

    return distances[end_router]
```

تعثيل شبكة (Topology) مكونة من 4 راوترات

(graph = {router: {neighbor: cost}})